BINUS UNIVERSITY ONLINE LEARNING

People Innovation Excellence

LECTURE NOTE

Week 07

Dynamic Programming

Multistage Graph and Travelling Salesman Problem

ALGORITHM DESIGN AND ANALYSIS

LEARNING OUTCOMES

LO2: Compare several algorithm design methods

OUTLINE MATERI :

1. Dynamic Programming: Multistage Graph
2. Dynamic Programming: Travelling Salesman Problem

ISI MATERI

1. Dynamic Programming: Multistage Graph Problem

Graf bertingkat $\boldsymbol{G} =< \boldsymbol{V}, \boldsymbol{C} >$ adalah graf berarah dimana simpul-simpulnya dipartisi menjadi $\boldsymbol{k} \geq \mathbf{2}$, himpunan terputus $\boldsymbol{V_i}$, di mana $\mathbf{1} \leq \boldsymbol{i} \leq \boldsymbol{n}$. Setiap edge menghubungkan dua node dari dua partisi. Setiap edge diasosiasikan dengan edge cost $\boldsymbol{C}(\boldsymbol{i}, \boldsymbol{j})$. Node awal disebut source dan node terakhir disebut sink. Masalah multistage adalah masalah menemukan jalur terpendek dari sumber ke tujuan. Masalah ini juga disebut 'masalah pelatih panggung'. Pendekatan pemrograman dinamis dapat digunakan untuk memecahkan masalah ini.

Menerapkan pendekatan pemrograman dinamis, masalah dibagi menjadi submasalah, yang disebut tahapan. Misalnya, memiliki lima tahap, tahap 1 dengan simpul {1}, tahap 2 dengan simpul {2,3,4,5}, tahap 3 dengan simpul {6,7,8}, tahap 4 dengan simpul {9, 10,11}, dan terakhir tahap 5 dengan simpul {12}. Pemrograman dinamis dimulai dengan masalah kecil. Masalah yang lebih kecil adalah perjalanan yang hampir selesai dengan hanya satu tahap lagi untuk beralih dari keadaan saat ini ke tujuan. Oleh karena itu, awalnya jalur terpendek dari tahap 4 ke tahap 5 dihitung. Kemudian masalah diperbesar dengan menambahkan satu tahap lagi ke masalah saat ini. Dalam hal ini menambahkan satu tahap lagi ke tahap yang belum selesai untuk mencapai tujuan. Dapat diamati bahwa tidak perlu penghitungan ulang, karena biaya tahap yang sudah disimpan dapat digunakan kembali. Dengan cara ini, seseorang dapat menemukan jalur terpendek dari 1 hingga 12. Jalur terpendek dapat diperoleh dengan menggunakan prosedur komputasi maju dan mundur. Pada prosedur komputasi maju, keputusan $\boldsymbol{x_i}$ dibuat dalam bentuk keputusan optimal $(\boldsymbol{x_{n-1}}, \boldsymbol{x_{n-1}}, \ldots, \boldsymbol{x_1})$, sedangkan pada prosedur komputasi mundur, keputusan $\boldsymbol{x_i}$ dibuat dalam bentuk optimal keputusan $(\boldsymbol{x_1}, \boldsymbol{x_2}, \boldsymbol{x_3}, \ldots, \boldsymbol{x_{i-1}})$.

Multistage graph berbasis metode dynamic programming dibagi menjadi 2 metode, yakni metode forward dan metode backward.

1) Forward method

Algoritma informal untuk prosedur komputasi forward method untuk grafik multistage diberikan sebagai berikut:

step 1: Read directed graph $\boldsymbol{G} =< \boldsymbol{V}, \boldsymbol{C} >$ with k stages

step 2: Let n be the number of nodes and dist [1..k] be the distance array

step 3: set initial cost to zero

step 4: Loop index i from (n-1) to 1

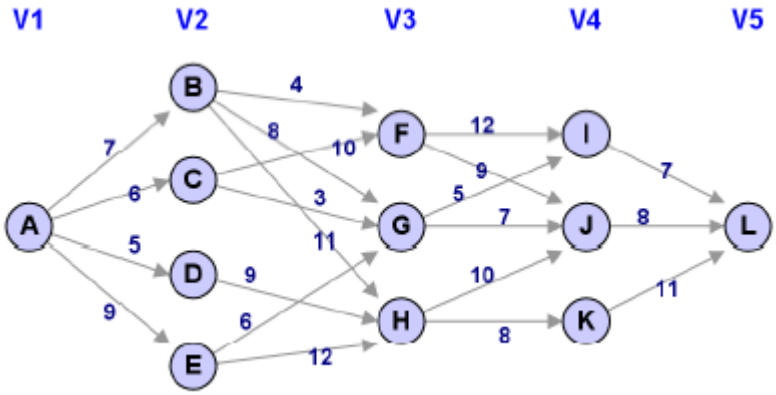
4a: Find a vertex v from the next stage such that the edge connecting the current stage and the next stage is minimum, that is (j,v) + cost(v) is minimum.

4b: Update cost and store v in another array dist[]

step 5: return cost

step 6: end

Perhatikan multistage graph berikut:



Sekarang kita akan mencoba Menggunakan metode forward untuk mencari solusi jalur terpendek dari node A ke node L.

Metode forward berbasis pada formula berikut:

cost(i,j) = min{c(j, k) + cost(i+1, k)}

dimana:

i = level dari node tersebut.

j = node awal/ node sumber

k = node akhir / node Tujuan

c(j,k) = jarak dari node sumber ke node Tujuan

cost(i, j) = fungsi untuk mencari jalur terpendek dimana fungsi ini bersifat rekursif. Untuk level **i** dengan node sumber **j**.

Keterangan:

Metode ini Menggunakan konsep dari dynamic programming. Karena metode ini mencari solusi berdasarkan subset solusinya dan hanya mengerjakan pola yang memungkinkan saja.

Penyelesaiannya:

Cost(5,L) = 0

Cost(4,I) = min {c(I, L) + cost(5.L)} = 7

cost(4,J) = min { c(J,L) + cost(5,L) } = 8

cost(4,K) = min { c(K,L) + cost(5,L) } = 11

cost(3,F) = min { c(F,I) + cost(4,I) | c(F,J) + cost(4,J) }

cost(3,F) = min { 12 + 7 | 9 + 8 } = 17

cost(3,G) = min { c(G,I) + cost(4,I) | c(G,J) + cost(4,J) }

cost(3,G) = min { 5 + 7 | 7 + 8 } = 12

cost(3,H) = min { c(H,J) + cost(4,J) | c(H,K) + cost(4,K) }

cost(3,H) = min { 10 + 8 | 8 + 11 } = 18

cost(2,B) = min {

c(B,F) + cost(3,F) |

c(B,G) + cost(3,G) |

c(B,H) + cost(3,H)

}

cost(2,B) = min { 4 + 17 | 8 + 12 | 11 + 18 } = 20

cost(2,C) = min { c(C,F) + cost(3,F) | c(C,G) + cost(3,G) }

cost(2,C) = min { 10 + 17 | 3 + 12 } = 15

cost(2,D) = min { c(D,H) + cost(3,H) }

cost(2,D) = min { 9 + 18 } = 27

cost(2,E) = min { c(E,G) + cost(3,G) | c(E,H) + cost(3,H) }

cost(2,E) = min { 6 + 12 | 12 + 18 } = 18

cost(1,A) = min {

c(A,B) + cost(2,B) |

c(A,C) + cost(2,C) |

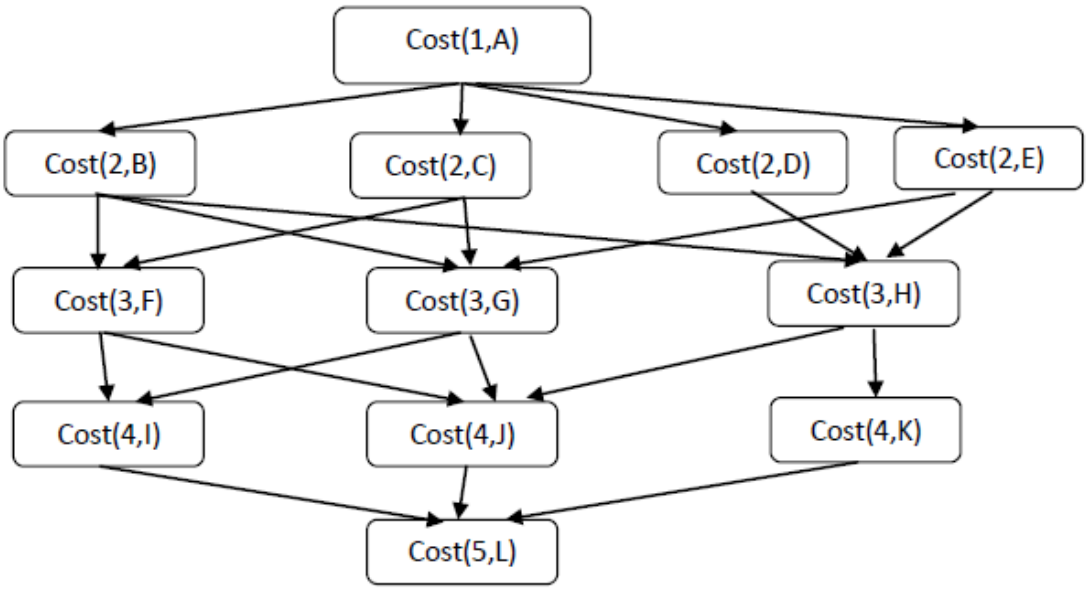
c(A,D) + cost(2,D) |

c(A,E) + cost(2,E)

}

cost(1,A) = min { 7 + 20 | 6 + 15 | 5 + 27 | 9 + 18 } = 21

Metode forward dimulai dengan pemanggilan fungsi cost(1,A) yang berjalan secara maju ke depan. Prosesnya akan berjalan seperti berikut:



- Ketika pemanggilan fungsi cost(1,A) maka fungsi tersebut akan memanggil fungsi lain yang menandakan hubungan adanya jalur yang terhubung antar node. Seperti layaknya fungsi rekursif lainnya, fungsi ini baru akan selesai dijalankan apabila proses didalamnya telah selesai dijalankan. Fungsi – fungsi yang dijalankan antara lain cost(2,B), cost(2,C), cost(2,D), dan cost(2,E) pemanggilan tersebut akan disertai dengan penambahan bobot jalur ke masing – masing node yang dijabarkan pada fungsi tersebut sehingga nantinya akan didapatkan bobot jalur dari node awal, yakni A ke node akhir melalui jalur yang ada. Dari hasil tersebut akan diambil nilai yang paling minimum.

- Ketika pemanggilan fungsi cost(2,B) maka fungsi tersebut akan memanggil fungsi lainnya, yakni cost(3,F), cost(3,G), dan cost(3,H) pemanggilan fungsi ini akan mengembalikan bobot terpendek menuju node akhir dari node specific yang tergambarkan pada pemanggilan fungsi dan akan ditambahkan dengan bobot jarak dari node B ke masing - masing node yang dituju. Fungsi ini nantinya akan mengembalikan nilai yang akan diterima oleh fungsi pemanggilnya, yakni cost(1,A).

- Ketika pemanggilan fungsi cost(2,C) maka fungsi tersebut akan memanggil fungsi lainnya, yakni cost(3,F) dan cost(3,G) pemanggilan fungsi ini akan mengembalikan bobot terpendek menuju node akhir dari node specific yang tergambarkan pada pemanggilan fungsi dan akan ditambahkan dengan bobot jarak dari node C ke masing - masing node yang dituju. Fungsi ini nantinya akan mengembalikan nilai yang akan diterima oleh fungsi pemanggilnya, yakni cost(1,A).

- Ketika pemanggilan fungsi cost(2,D) maka fungsi tersebut akan memanggil fungsi lainnya, yakni cost(3,H) pemanggilan fungsi ini akan mengembalikan bobot terpendek menuju node akhir dari node specific yang tergambarkan pada pemanggilan fungsi dan akan ditambahkan dengan bobot jarak dari node D ke masing - masing node yang dituju. Fungsi ini nantinya akan mengembalikan nilai yang akan diterima oleh fungsi pemanggilnya, yakni cost(1,A).

- Ketika pemanggilan fungsi cost(2,E) maka fungsi tersebut akan memanggil fungsi lainnya, yakni cost(3,G) dan cost(3,H) pemanggilan fungsi ini akan mengembalikan bobot terpendek menuju node akhir dari node specific yang tergambarkan pada pemanggilan fungsi dan akan ditambahkan dengan bobot jarak dari node E ke masing - masing node yang dituju. Fungsi ini nantinya akan mengembalikan nilai yang akan diterima oleh fungsi pemanggilnya, yakni cost(1,A).

- Ketika pemanggilan fungsi cost(3,F) maka fungsi tersebut akan memanggil fungsi lainnya, yakni cost(4,I) dan cost(4,J) pemanggilan fungsi ini akan mengembalikan bobot terpendek menuju node akhir dari node specific yang tergambarkan pada pemanggilan fungsi dan akan ditambahkan dengan bobot jarak dari node F ke masing - masing node yang dituju. Fungsi ini nantinya akan mengembalikan nilai yang akan diterima oleh fungsi pemanggilnya.

- Ketika pemanggilan fungsi cost(3,G) maka fungsi tersebut akan memanggil fungsi lainnya, yakni cost(4,I) dan cost(4,J) pemanggilan fungsi ini akan mengembalikan bobot terpendek menuju node akhir dari node specific yang tergambarkan pada pemanggilan fungsi dan akan ditambahkan dengan bobot jarak dari node G ke masing - masing node

yang dituju. Fungsi ini nantinya akan mengembalikan nilai yang akan diterima oleh fungsi pemanggilnya.

- Ketika pemanggilan fungsi cost(3,H) maka fungsi tersebut akan memanggil fungsi lainnya, yakni cost(4,J) dan cost(4,K) pemanggilan fungsi ini akan mengembalikan bobot terpendek menuju node akhir dari node specific yang tergambarkan pada pemanggilan fungsi dan akan ditambahkan dengan bobot jarak dari node H ke masing - masing node yang dituju. Fungsi ini nantinya akan mengembalikan nilai yang akan diterima oleh fungsi pemanggilnya.

- Ketika pemanggilan fungsi cost(4,I), cost(4,J) dan cost (4,K) maka fungsi tersebut mengecek level dari node yang dituju dikarenakan node yang dituju telah sama dengan level tertinggi maka akan diambil jarak masing-masing dari node tersebut ke node akhir. Fungsi ini nantinya akan mengembalikan nilai yang akan diproses oleh fungsi pemanggilnya.

- Ketika pemanggilan fungsi cost(5,L). Karena level pada pemanggilan fungsi ini merupakan level tertinggi maka nilai yang akan dibalikan adalah 0.

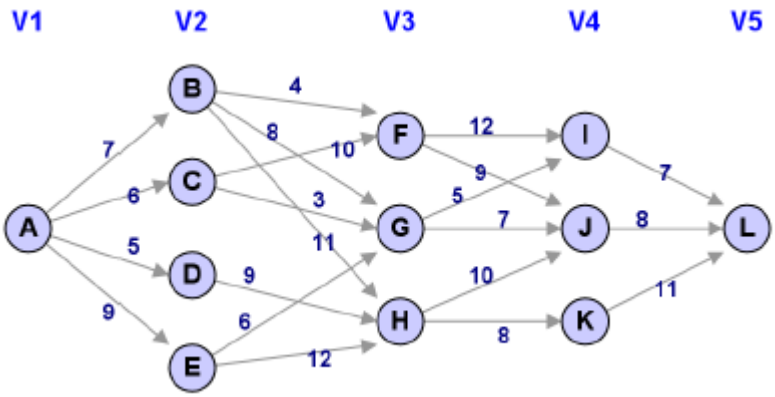
- Dari proses diatas terlihat dengan jelas bahwa metode ini menerapkan konsep dari dynamic programming dimana untuk menyelesaikan masalahnya akan diselesaikan dengan penyelesain masalah lain. Hal ini akan bersifat dinamis dengan jumlah node dan level sebanyak apapun.

- Metode ini memiliki banyak ragam algoritma untuk menyelesaikannya. Namun polanya akan selalu sama.

2) Backward method

Seperti yang disampaikan sebelumnya, multistage graph dapat juga diselesaikan menggunakan prosedur komputasi backward. Backward method sama dengan forward method tapi hanya beda satu aspek yaitu loop track dari 1 ke n – 1.

Perhatikan multistage graph berikut:



Sekarang kita akan mencoba Menggunakan metode backward untuk mencari solusi jalur terpendek dari node A ke node L.

Metode backward berbasis pada formula berikut:

bcost(i,j) = min{(l,j) + bcost(i-1,l)}

Dimana:

i = level dari node tersebut.

l = node awal / node sumber (menunjuk node sebelumnya / node yang mengarah ke node j)

j = node akhir / node tujuan (posisi node yang ditunjuk sekarang)

c(l,j) = jarak dari node sumber ke node tujuan

bcost(i,j) = fungsi untuk mencari jalur terpendek dimana fungsi ini bersifat rekursif untuk level i dengan node sumber j.

Keterangan:

Metode ini menggunakan konsep dari dynamic programming. Karena metode ini mencari solusi berdasarkan subset solusinya dan hanya mengerjakan pola yang memungkinkan saja.

Penyelesaiannya:

bcost(1,A) = 0

bcost(2,B) = min { c(A,B) + bcost(1,A) } = 7

bcost(2,C) = min { c(A,C) + bcost(1,A) } = 6

bcost(2,D) = min { c(A,D) + bcost(1,A) } = 5

bcost(2,E) = min { c(A,E) + bcost(1,A) } = 9.

bcost(3,F) = min { c(B,F) + bcost(2,B) | c(C,F) + bcost(2,C) }

bcost(3,F) = min { 4 + 7 | 10 + 6 } = 11

bcost(3,G) = min {

c(B,G) + bcost(2,B) |

c(C,G) + bcost(2,C) |

c(E,G) + bcost(2,E)

}

bcost(3,G) = min { 8 + 7 | 3 + 6 | 6 + 9 } = 9

bcost(3,H) = min {

c(B,H) + bcost(2,B) |

c(D,H) + bcost(2,D) |

c(E,H) + bcost(2,E)

}

bcost(3,H) = min { 11 + 7 | 9 + 5 | 12 + 9 } = 14

bcost(4,I) = min { c(F,I) + bcost(3,F) | c(G,I) + bcost(3,G) }

bcost(4,I) = min { 12 + 11 | 5 + 9 } = 14

bcost(4,J) = min {

c(F,J) + bcost(3,F) |

c(G,J) + bcost(3,G) |

c(H,J) + bcost(3,H)

}

bcost(4,J) = min { 9 + 11 | 7 + 9 | 10 + 14 } = 16

bcost(4,K) = min { c(H,K) + cost(3,H) }

bcost(4,K) = min { 8 + 14 } = 22

bcost(5,L) = min {

c(I,L) + bcost(4,I) |

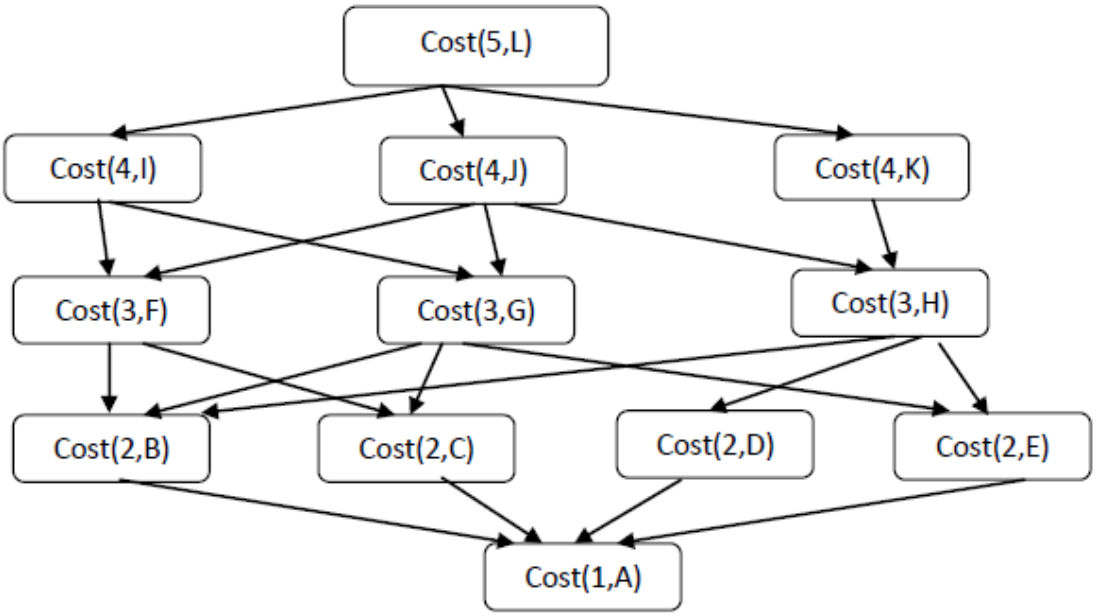
c(J,L) + bcost(4,J) |

c(K,L) + bcost(4,K)

}

bcost(5,L) = min { 7 + 14 | 8 + 16 | 11 + 22 } = 21

Metode backward dimulai dengan pemanggilan fungsi bcost(5,L) yang berjalan mundur ke belakang. Prosesnya akan berjalan seperti berikut:



- Ketika pemanggilan fungsi bcost(5,L) maka fungsi tersebut akan memanggil fungsi lain yang menandakan hubungan adanya jalur yang terhubung antar node. Seperti layaknya fungsi rekursif lainnya, fungsi ini baru akan selesai dijalankan apabila proses didalamnya telah selesai dijalankan. Fungsi – fungsi yang dijalankan antara lain bcost(4,I), bcost(4,J), dan bcost(4,K) pemanggilan tersebut akan disertai dengan penambahan bobot jalur ke masing – masing node yang dijabarkan pada fungsi tersebut sehingga nantinya akan didapatkan bobot jalur dari node awal, yakni I ke L, J ke L, dan K ke L. Dari hasil tersebut akan diambil nilai yang paling minimum.

- Ketika pemanggilan fungsi bcost(4,I) maka fungsi tersebut akan memanggil fungsi lainnya, yakni bcost(3,F), bcost(3,G), dan bcost(3,H) pemanggilan fungsi ini akan mengembalikan bobot terpendek menuju node akhir dari node specific yang tergambarkan pada pemanggilan fungsi dan akan ditambahkan dengan bobot jarak menuju node I dari masing - masing node penunjuknya, yakni node F, node G, dan node H. Fungsi ini nantinya akan mengembalikan nilai yang akan diterima oleh fungsi pemanggilnya, yakni bcost(5,L).

- Ketika pemanggilan fungsi bcost(4,J) maka fungsi tersebut akan memanggil fungsi lainnya, yakni bcost(3,F), bcost(3,G), dan bcost(3,H) pemanggilan fungsi ini akan

mengembalikan bobot terpendek menuju node akhir dari node specific yang tergambarkan pada pemanggilan fungsi dan akan ditambahkan dengan bobot jarak menuju node J dari masing - masing node penunjuknya, yakni node F, node G, dan node H. Fungsi ini nantinya akan mengembalikan nilai yang akan diterima oleh fungsi pemanggilnya, yakni bcost(5,L).

- Ketika pemanggilan fungsi bcost(4,K) maka fungsi tersebut akan memanggil fungsi lainnya, yakni bcost(3,H) pemanggilan fungsi ini akan mengembalikan bobot terpendek menuju node akhir dari node specific yang tergambarkan pada pemanggilan fungsi dan akan ditambahkan dengan bobot jarak menuju node K dari masing - masing node penunjuknya, yakni node H. Fungsi ini nantinya akan mengembalikan nilai yang akan diterima oleh fungsi pemanggilnya, yakni bcost(5,L).

- Ketika pemanggilan fungsi bcost(3,F) maka fungsi tersebut akan memanggil fungsi lainnya, yakni bcost(2,B) dan bcost(2,C) pemanggilan fungsi ini akan mengembalikan bobot terpendek menuju node akhir dari node specific yang tergambarkan pada pemanggilan fungsi dan akan ditambahkan dengan bobot jarak menuju node F dari masing - masing node penunjuknya, yakni node B dan node C. Fungsi ini nantinya akan mengembalikan nilai yang akan diterima oleh fungsi pemanggilnya.

- Ketika pemanggilan fungsi bcost(3,G) maka fungsi tersebut akan memanggil fungsi lainnya, yakni bcost(2,B), bcost(2,C), dan bcost(2,E) pemanggilan fungsi ini akan mengembalikan bobot terpendek menuju node akhir dari node specific yang tergambarkan pada pemanggilan fungsi dan akan ditambahkan dengan bobot jarak menuju node G dari masing - masing node penunjuknya, yakni node B, node C, dan node E. Fungsi ini nantinya akan mengembalikan nilai yang akan diterima oleh fungsi pemanggilnya.

- Ketika pemanggilan fungsi bcost(3,H) maka fungsi tersebut akan memanggil fungsi lainnya, yakni bcost(2,B), bcost(2,D), dan bcost(2,E) pemanggilan fungsi ini akan mengembalikan bobot terpendek menuju node akhir dari node specific yang tergambarkan pada pemanggilan fungsi dan akan ditambahkan dengan bobot jarak

menuju node H dari masing - masing node penunjuknya, yakni node B, node D, dan node E. Fungsi ini nantinya akan mengembalikan nilai yang akan diterima oleh fungsi pemanggilnya.

- Ketika pemanggilan fungsi bcost(2,B), bcost(2,C), bcost(2,D) dan bcost(2,E) maka fungsi tersebut mengecek level dari node yang menuju ke masing-masing node yang dijabarkan dipemanggilan fungsi, yakni node B, node C, node D, dan node E. dikarenakan node yang dituju telah sama dengan level tertinggi maka akan diambil jarak masing-masing dari node tersebut ke node awal. Fungsi ini nantinya akan mengembalikan nilai yang akan diproses oleh fungsi pemanggilnya.

- Pemanggilan bcost(1,A) akan mengembalikan nilai 0 karena level dari node A berada pada level pertama.

- Sama halnya dengan mtedo forward, metode dapat dirancang dengan struktur algoritma yang berbeda namun tetap dengan pola yang sama.

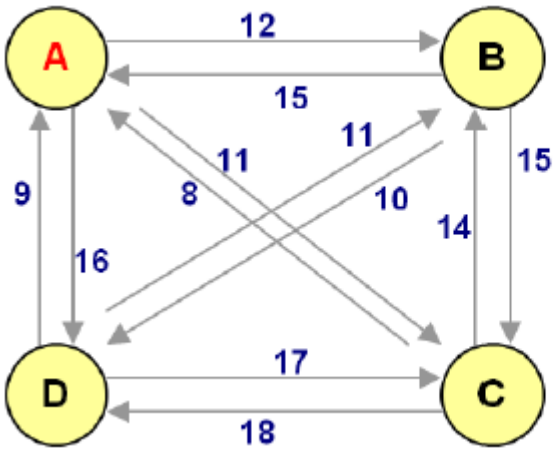
2. Dynamic Programming: Travelling Salesman Problem

Travelling salesman harus mulai dari satu titik dan melakukan perjalanan ke semua tempat dan kembali ke titik awal. Masalahnya adalah meminimalkan biaya perjalanan. Syarat utama disini adalah harus ada komunikasi antar node.

Misalkan kita harus merutekan van pos untuk mengambil surat dari kotak surat yang terletak di n situs yang berbeda. Grafik titik n + 1 dapat digunakan untuk mewakili situasi ini. Satu simpul mewakili kantor pos dari mana van pos dimulai dan ke mana ia harus kembali. Rute yang ditempuh oleh mobil pos adalah tur dan harus memiliki panjang minimum (biaya minimum).

Pada bagian ini kita akan membahas konsep serta Latihan dari permasalahan travelling salesman problem Menggunakan metode dynamic programming.

Perhatikan graph berikut:



$$\begin{bmatrix} \infty & 12 & 11 & 16 \\ 15 & \infty & 15 & 10 \\ 8 & 14 & \infty & 18 \\ 9 & 11 & 17 & \infty \end{bmatrix}$$

Untuk menyelesaikan permasalahan berikut, kita akan menerapkan formula berikut sebagai basis dari konsep dynamic programming.

$$p(i,L) = \min[c(j,i) + p(j,L-\{j\})]$$

dimana:

i = node yang akan dikunjungi

L = daftar / kumpulan node yang telah dikunjungi

j = node awal

c(j,i) = jarak dari node awal ke node yang akan dikunjungi

L-{j} = daftar / kumpulan node yang telah dikunjungi diluar node j

p(j, L-{j}) = fungsi mencari jalur terpendek untuk mengunjungi node j setelah mengunjungi node-node L dimana node j tidak termasuk dalam L.

p(i, L) = fungsi mencari jalur terpendek ke node i setelah mengunjungi node L.

p(B,Ø)=c(A,B)=12

- Pada bagian ini, Ø melambangkan bahwa belum ada node yang dikunjungi maka perjalanan akan dimulai dari node A. Karena node A adalah awal. Pada TSP titik mulai adalah titik selesai.
- Oleh Karena itu pada p(B, Ø) akan menghitung jalur dari A ke B.

p(C,Ø)=c(A,C)=11

- Pada bagian ini, Ø melambangkan bahwa belum ada node yang dikunjungi maka perjalanan akan dimulai dari node A. Karena node A adalah awal. Pada TSP titik mulai adalah titik selesai.
- Oleh Karena itu pada p(C, Ø) akan menghitung jalur dari A ke C.

p(D,Ø)=c(A,D)=16

- Pada bagian ini, Ø melambangkan bahwa belum ada node yang dikunjungi maka perjalanan akan dimulai dari node A. Karena node A adalah awal. Pada TSP titik mulai adalah titik selesai.
- Oleh Karena itu pada p(C, Ø) akan menghitung jalur dari A ke D.

p(B,{C})=c(C,B)+p(C,Ø)=25

- Pada bagian ini, {C} menandakan bahwa node C telah dikunjungi atau dengan kata lain bahwa jalur A ke C telah dikunjungi.
- Oleh Karena itu pada p(B, {C}) akan menghitung jalur dari A ke C dan dilanjutkan ke D. A – C – D.

p(B,{D})=c(D,B)+p(D,Ø)=27

- Pada bagian ini, {D} menandakan bahwa node D telah dikunjungi atau dengan kata lain bahwa jalur A ke D telah dikunjungi.
- Oleh Karena itu pada p(B, {D}) akan menghitung jalur dari A ke D dan dilanjutkan ke B. A – D – B.

p(C,{B})=c(B,C)+p(B,Ø)=29

- Pada bagian ini, {B} menandakan bahwa node B telah dikunjungi atau dengan kata lain bahwa jalur A ke B telah dikunjungi.
- Oleh Karena itu pada p(C, {B}) akan menghitung jalur dari A ke B dan dilanjutkan ke C. A – B – C.

p(C,{D})=c(D,C)+p(D,Ø)=33

- Pada bagian ini, {D} menandakan bahwa node D telah dikunjungi atau dengan kata lain bahwa jalur A ke D telah dikunjungi.
- Oleh Karena itu pada p(C, {D}) akan menghitung jalur dari A ke D dan dilanjutkan ke C. A – D – C.

p(D,{B})=c(B,D)+p(B,Ø)=22

- Pada bagian ini, {B} menandakan bahwa node B telah dikunjungi atau dengan kata lain bahwa jalur A ke B telah dikunjungi.
- Oleh Karena itu pada p(D, {B}) akan menghitung jalur dari A ke B dan dilanjutkan ke C. A – B – D.

p(D,{C})=c(C,D)+p(C,Ø)=29

- Pada bagian ini, {C} menandakan bahwa node C telah dikunjungi atau dengan kata lain bahwa jalur A ke C telah dikunjungi.
- Oleh Karena itu pada p(D, {C}) akan menghitung jalur dari A ke C dan dilanjutkan ke D. A – C – D.

p(B,{C,D}) = min[c(C,B)+p(C,{D})|c(D,B)+p(D,{C})]

= min[14+33|11+29] = 40

- Pada bagian ini, {C,D} menandakan bahwa beberapa jalur akan dikunjungi, yakni
  - ➔ A – C – D. => p(D,{C})
  - ➔ A – D – C. => p(C,{D})
  - ➔ Kedua jalur ini nantinya akan dilanjutkan menuju ke node B. => c(D,B)+p(D,{C}) dan c(C,B)+p(C,{D})
  - ➔ Hasil minimum perbandingan dari kedua jalur tersebut akan diambil sebagai jalur terpendek.
- Jadi, hasil dari fungsi ini adalah menemukan jalur terpendek dari 2 kemungkinan jalur, yakni jalur A – D – C – B dan A – C – D – B.

p(C,{B,D}) = min[c(B,C)+p(B,{D})|c(D,C)+p(D,{B})]

= min[15+27|17+22] = 39

- Pada bagian ini, {B,D} menandakan bahwa beberapa jalur akan dikunjungi, yakni
  - ➔ A – B – D. => p(D,{B})
  - ➔ A – D – B. => p(B,{D})
  - ➔ Kedua jalur ini nantinya akan dilanjutkan menuju ke node C. => c(B,C)+p(B,{D}) dan c(D,C)+p(D,{B})
  - ➔ Hasil minimum perbandingan dari kedua jalur tersebut akan diambil sebagai jalur terpendek.
- Jadi, hasil dari fungsi ini adalah menemukan jalur terpendek dari 2 kemungkinan jalur, yakni jalur A – D – B – C dan A – B – D – C.

p(D,{B,C}) = min[c(B,D)+p(B,{C})|c(C,D)+p(C,{B})]

= min[10+25|18+27] = 35

- Pada bagian ini, {B,C} menandakan bahwa beberapa jalur akan dikunjungi, yakni
  - ➔ A – B – C. => p(C,{B})
  - ➔ A – C – B. => p(B,{C})
  - ➔ Kedua jalur ini nantinya akan dilanjutkan menuju ke node D. => c(B,D)+p(B,{C}) dan c(C,D)+p(C,{B})
  - ➔ Hasil minimum perbandingan dari kedua jalur tersebut akan diambil sebagai jalur terpendek.
- Jadi, hasil dari fungsi ini adalah menemukan jalur terpendek dari 2 kemungkinan jalur, yakni jalur A – C – B – D dan A – B – C – D.

p(A,{B,C,D}) =min[c(B,A)+p(B,{C,D})|c(C,A)+p(C,{B,D})|c(D,A)+p(D, {B,C})]

= min[15+40|8+39|9+35] = 44

- Pada bagian ini, {B,C,D} menandakan bahwa beberapa jalur akan dikunjungi, yakni
  - ➔ A – B – C – D dan A – C – B – D.

    => p(D,{B,C})
  - ➔ A – B – D – C dan A – D – B – C.

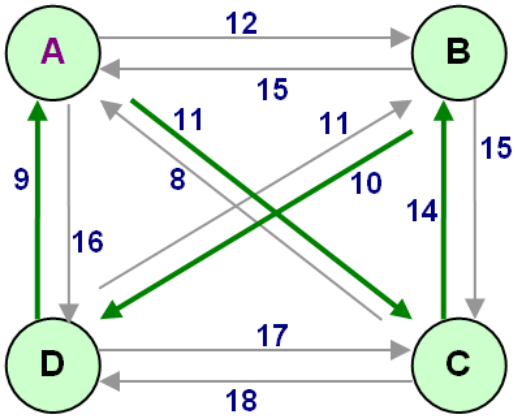
    => p(C,{B,D})
  - ➔ A – C – D – B dan A – D – C – B.

    => p(B,{C,D})
  - ➔ Jalur – jalur ini nantinya akan dilanjutkan menuju kembali ke node A.

    => c(B,A)+p(B,{C,D}); c(C,A)+p(C,{B,D}); dan c(D,A)+p(D,{B,C})
  - ➔ Hasil minimum perbandingan dari ketiga jalur tersebut akan diambil sebagai jalur terpendek.

- Jadi, hasil dari fungsi ini adalah menemukan jalur terpendek dari node A ke semua node lain dan kembali ke node A.

  Hasil perhitungan jalur terpendek untuk permasalahan TSP ini adalah **44** dengan jalur **c(D,A)+p(D, {B,C}).**



KESIMPULAN

Masalah multistage adalah masalah menemukan jalur terpendek dari sumber ke tujuan. Masalah ini juga disebut 'masalah pelatih panggung'. Pendekatan pemrograman dinamis dapat digunakan untuk memecahkan masalah ini. Multistage graph berbasis metode dynamic programming dibagi menjadi 2 metode, yakni metode forward dan metode backward.

Travelling salesman harus mulai dari satu titik dan melakukan perjalanan ke semua tempat dan kembali ke titik awal. Masalahnya adalah meminimalkan biaya perjalanan. Syarat utama disini adalah harus ada komunikasi antar node.

DAFTAR PUSTAKA

Sridhar, S. (2015). *Design and Analysis of Algorithms*. Oxford University Press.

MultiStage Graph - Dynamic Programming, www.youtube.com/watch?v=9iE9Mj4m8jk